Chapter 15 modelos de previsão de fundamentos

- Carregar bibliotecas e configurações comuns
- Carregar bibliotecas adicionais

15.1 Introdução

Nos últimos anos, os avanços em Redes Neurais (RNs), particularmente os modelos baseados em transformadores, têm atraído atenção na previsão de séries temporais. Com base em trabalhos de outras áreas, a pesquisa na aplicação de RNs a tarefas de séries temporais tem crescido rapidamente. Como explicado no Capítulo 14 , o principal atrativo reside na sua capacidade de extrair automaticamente representações significativas a partir de dados brutos, permitindo-lhes capturar relações não lineares complexas em dependências temporais.

Transformadores e Mecanismos de Atenção

Um dos desenvolvimentos mais influentes em aprendizado profundo foi a arquitetura Transformer, introduzida em Attention Is All You Need (Vaswani et al., 2017) . Ao contrário das redes neurais recorrentes (RNNs) ou das redes de memória de longo prazo (LSTM) tradicionais, os Transformers processam sequências de entrada inteiras em paralelo, em vez de sequencialmente. Isso elimina a necessidade de recorrência passo a passo, tornando-os mais escaláveis e eficientes para grandes conjuntos de dados.

A principal inovação dos Transformers reside no **mecanismo de autoatenção** , que permite ao modelo ponderar a importância de diferentes elementos de entrada, mesmo quando estes se encontram distantes numa sequência. Isto torna os Transformers particularmente eficazes na captura de dependências de longo alcance, uma vantagem crucial em domínios como o processamento de linguagem natural (PLN) e a previsão de séries temporais. Ao utilizar múltiplas camadas de redes de autoatenção e de propagação direta, os Transformers aprendem representações hierárquicas complexas dos dados, tornando-os ferramentas poderosas para diversas tarefas sequenciais.

Modelos de fundação e seu impacto

Os modelos fundamentais, incluindo os grandes modelos de linguagem (LLMs) no processamento de linguagem natural, transformaram a IA ao alcançarem desempenho de ponta em diversas aplicações. Esses modelos utilizam extensos conjuntos de dados para construir representações generalizadas, permitindo que tenham um bom desempenho de forma **"zero-shot"** — fazendo previsões sem serem especificamente treinados em um conjunto de dados em particular. Esse paradigma elimina a necessidade de desenvolver, implementar e treinar modelos específicos para cada caso de uso.

O conceito de modelos de base ganhou destaque com o BERT (Devlin et al., 2019) e o GPT (Radford e Narasimhan, 2018) , mas foi o GPT-3 (Brown et al., 2020) que demonstrou todo o seu potencial. Ao contrário dos modelos anteriores, que exigiam aprendizado supervisionado específico para cada tarefa, os modelos de base seguem uma **estrutura de pré-treinamento semissupervisionado** , na qual primeiro aprendem padrões gerais a partir de grandes quantidades de dados não rotulados e, em seguida, adaptam-se a tarefas específicas por meio de ajuste fino.

Embora essa abordagem tenha revolucionado o PNL (Processamento de Linguagem Natural), ela foi posteriormente estendida a outros domínios. Em visão computacional, modelos como o Vision Transformers (Dosovitskiy et al., 2021)

aplicaram arquiteturas baseadas em transformadores ao processamento de imagens. Em aplicações científicas, modelos fundamentais como o AlphaFold (Jumper et al., 2021) avançaram significativamente a predição da estrutura de proteínas.

Dado o sucesso dos modelos fundamentais no processamento de linguagem natural e na visão computacional, os pesquisadores têm explorado sua aplicação na previsão de séries temporais. Um trabalho inicial notável (Oreshkin et al., 2021) demonstrou que o modelo N-BEATS poderia ser pré-treinado para permitir a previsão zero-shot.

Vantagens potenciais

Os modelos de base oferecem benefícios importantes em relação às abordagens tradicionais específicas para tarefas na previsão de séries temporais. Embora essa tecnologia ainda esteja em estágios iniciais, suas vantagens incluem:

- **Facilidade de uso** : Os modelos básicos são pré-treinados em conjuntos de dados de grande escala, permitindo a aplicação direta a novos conjuntos de dados sem treinamento extensivo.
- **Lower computational requirements**: They reduce computational costs by eliminating intensive training on individual datasets.
- **Accuracy with limited data**: Foundation models excel with limited data. Their extensive pre-training helps capture underlying patterns that data-specific models often miss with limited training data.

Overall, foundation models represent a significant shift in time series forecasting, providing a flexible and scalable alternative to traditional methods. As research continues to evolve, these models have the potential to redefine forecasting across various industries.

15.2 Transfer learning

Foundation models rely on their capabilities to generalize across domains, particularly in new tasks that were not available during training. We understand, accordingly, transfer learning as the capacity to apply knowledge gleaned from one task to solve new tasks.

A forecasting model provides a function $f_\theta : X \mapsto Y$, with X the feature space and Y the dependent variable space. We consider the setting with $X = \{y_{[0:t]}, x_{[0:t+h]}\}$ and $Y = \{y_{[t+1:t+h]}\}$, where h is the forecast horizon, y is the target time series, and x are exogenous covariates. The forecasting task objective is to estimate the following conditional distribution:

$$P(y_{[t+1:t+h]}|y_{[0:t]}, x_{[0:t+h]}) = f_\theta(y_{[0:t]}, x_{[0:t+h]}) \tag{1}$$

Transfer-learning refers to pre-training a model on a (usually large) source dataset to improve its performance on a new forecasting task with a target dataset.

The core idea of a foundation model is to leverage this principle by training it on a large time series dataset, leveraging scaling laws on the dataset and model sizes. A diverse dataset, in terms of breadth and depth, is necessary for the model to perform in a wide variety of domains.

Example: Transfer-learning from M4 to Air Passengers

In the following simple example we illustrate this process by pre-training the NHITS (Challu et al. 2023) on the M4 monthly dataset. The M4 dataset is a collection of 100,000 time series used for the fourth edition of the Makridakis forecasting Competition (Makridakis, Spiliotis, and Assimakopoulos 2020). The M4 dataset consists of time series of yearly, quarterly, monthly and other (weekly, daily and hourly) data, which are divided into training and test sets. The M4 dataset was created by selecting a random sample of 100,000 time series from the ForeDeCk database. The selected series were then scaled to prevent negative observations and values lower than 10, thus avoiding possible problems when calculating various error measures. The scaling was performed by simply adding a constant to the series so that

their minimum value was equal to 10 (29 occurrences across the whole dataset). In addition, any information that could possibly lead to the identification of the original series was removed so as to ensure the objectivity of the results.

```
Y_df = M4.load(directory="./", group="Monthly")[0].assign(
    ds=lambda df: df.groupby("unique_id")["ds"].transform(lambda x:
        pd.date_range(start="1970-01-01", periods=len(x), freq="MS")
    )
)
Y_df.head()
```

	unique_id	ds	y
0	M1	1970-01-01	8000
1	M1	1970-02-01	8350
2	M1	1970-03-01	8570
3	M1	1970-04-01	7700
4	M1	1970-05-01	7080

```
horizon = 12
stacks = 3
models = [
    NHITS(
        input_size=5 * horizon,
        h=horizon,
        max_steps=2_000,
        stack_types=stacks * ["identity"],
        n_blocks=stacks * [1],
        mlp_units=[[256, 256] for _ in range(stacks)],
        n_pool_kernel_size=stacks * [1],
        batch_size=32,
        scaler_type="standard",
        n_freq_downsample=[12, 4, 1],
    )
]
nf = NeuralForecast(models=models, freq="MS")
nf.fit(df=Y_df)
```

Then, we can use the pre-trained model to forecast the `air_passengers` dataset, unseen during training.

```
air_passengers_df = AirPassengersDF.copy()
transfer_preds = nf.predict(df=air_passengers_df)
```

```
plot_series(air_passengers_df, transfer_preds,
    xlabel="Month", ylabel="Number of passengers",
    title = "International airline passengers",
    palette="black_and_blue", rm_legend=False)
```

Figure 15.1: Forecasts by transfer learning.

In the figure above, we can see that the model, despite not being trained on this particular dataset, is able to generate forecasts that capture the overall behavior of the series.

15.3 Overview of foundation models

Foundation models represent a paradigm shift in the field of forecasting, enabling the application of pre-trained models to a wide range of time series tasks without the need for dataset-specific training. This approach, still in its early stages, has generated significant attention, leading to the development of multiple models. Below is a chronological overview of notable foundation models:

- Time-LLM (Jin et al. 2023)
- TimeGPT-1 (Garza, Challu, and Mergenthaler-Canseco 2023)
- Lag-Llama (Rasul et al. 2023)
- TimesFM (Das et al. 2023)
- Tiny Time Mixers (Ekambaram et al. 2024)
- Moirai (Woo et al. 2024)
- MOMENT (Goswami et al. 2024)
- UniTS (Gao et al. 2024)
- Chronos (Ansari et al. 2024)

Next, we present an overview of some of these models. Understanding the core components of the transformer is required to understand the particularities of each large time model. We suggest you read The Illustrated Transformer if you are not comfortable with some of the following terms and concepts.

TimeGPT

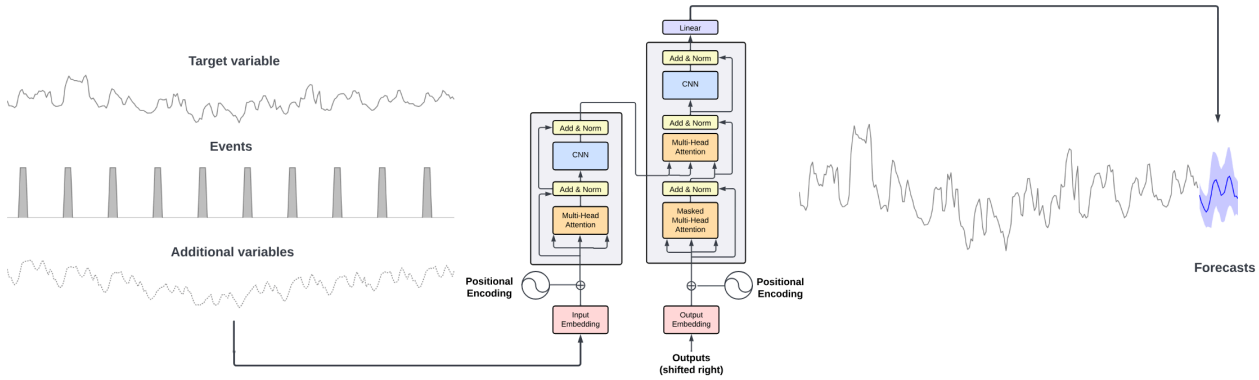

Figure 15.2: Architecture of TimeGPT.

TimeGPT is a transformer-based time series model that uses self-attention mechanisms, a method that allows the model to weigh the importance of different past observations when making a forecast. Instead of treating all past values equally, self-attention assigns different levels of importance based on how relevant each point is to the current prediction. TimeGPT takes a window of historical values as input and enhances it with local positional encoding, a technique that provides information about the order of data points—critical for understanding time-dependent patterns. The architecture follows an encoder-decoder structure, where the encoder processes input time series and extracts relevant features, while the decoder uses this processed information to generate forecasts. Each layer in the model includes residual connections (which help preserve information and improve training stability) and layer normalization (which standardizes activations to ensure stable learning). Finally, a linear layer maps the decoder's output to match the desired forecast horizon. The general intuition behind attention-based models is their ability to learn from diverse past events and extrapolate potential future distributions based on patterns observed in historical data.

TimeGPT was trained on a large collection of publicly available time series, collectively encompassing over 100 billion data points. This training set incorporates time series from a broad array of domains, including finance, economics, demographics, healthcare, weather, IoT sensor data, energy, web traffic, sales, transport, and banking.

For more information:

- read the original paper
- see the official repository

Lag-Llama

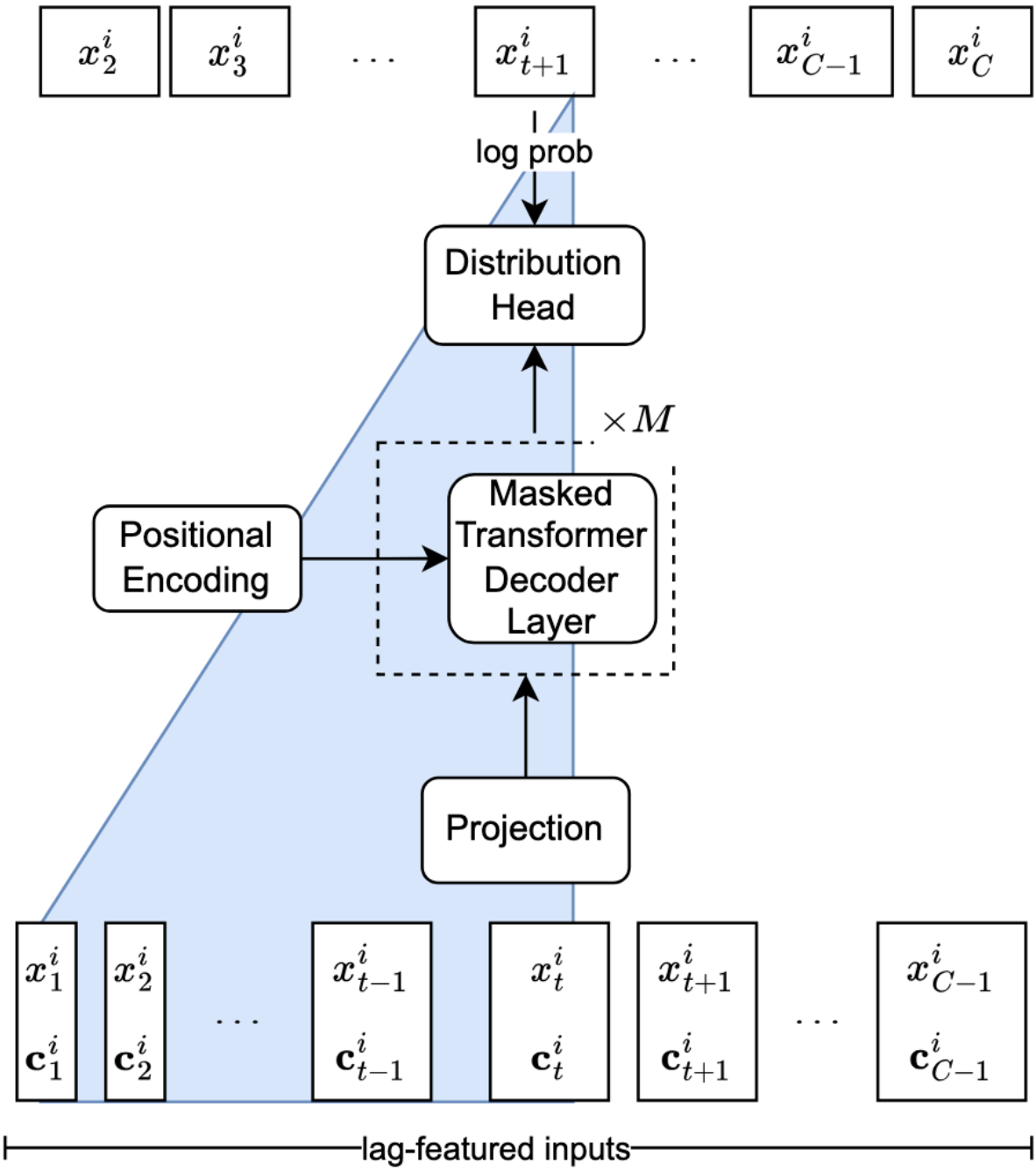

Figure 15.3: Architecture of Lag-Llama.

Lag-Llama is a probabilistic univariate forecasting model. It is a decoder-only transformer, meaning it does not use an encoder to preprocess the data; instead, it directly generates the forecast. The model builds lagged features, which are previous time series values that serve as inputs to the model. For example, it extracts seasonal indicators such as the quarter, month, day of the week, and finer-grained time elements down to the second. This helps the model capture seasonal patterns and short-term fluctuations. To estimate uncertainty in its predictions, Lag-Llama assumes a Student's t-distribution.

For more information:

- read the original paper
- see the official repository

TimesFM

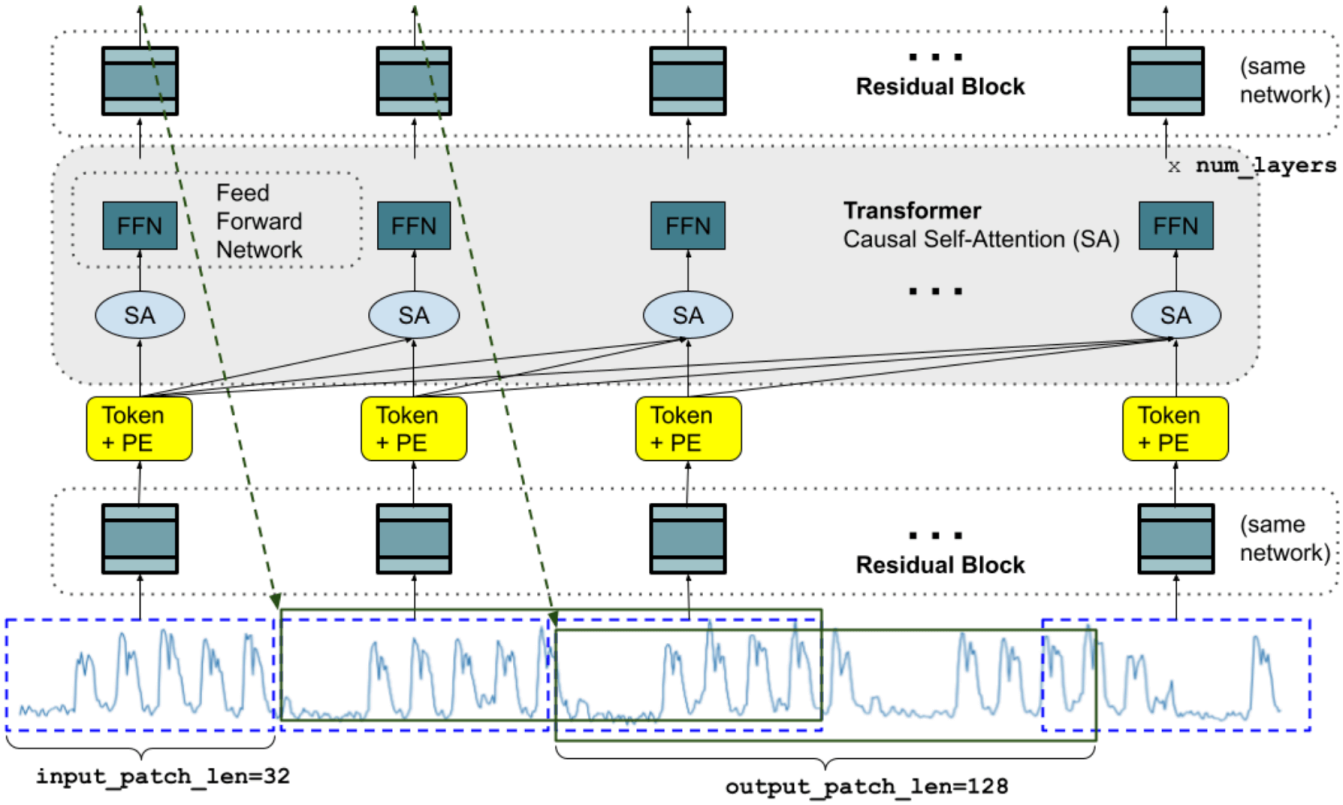

Figure 15.4: Architecture of TimesFM.

TimesFM is decoder-based foundation model developed by Google Research. It is a deterministic model focusing on point forecasts. It relies on patching the series to extract local semantic information from groups of data points. The model patches the input series and is trained to output longer output patches, such that it undergoes fewer autoregressive steps if the forecast horizon exceeds the output patch length. It uses a decoder-only transformer and uses a residual block to embed the input series and map the output of the decoder to predictions.

For more information:

- read the original paper
- see the official repository

Tiny Time Mixers

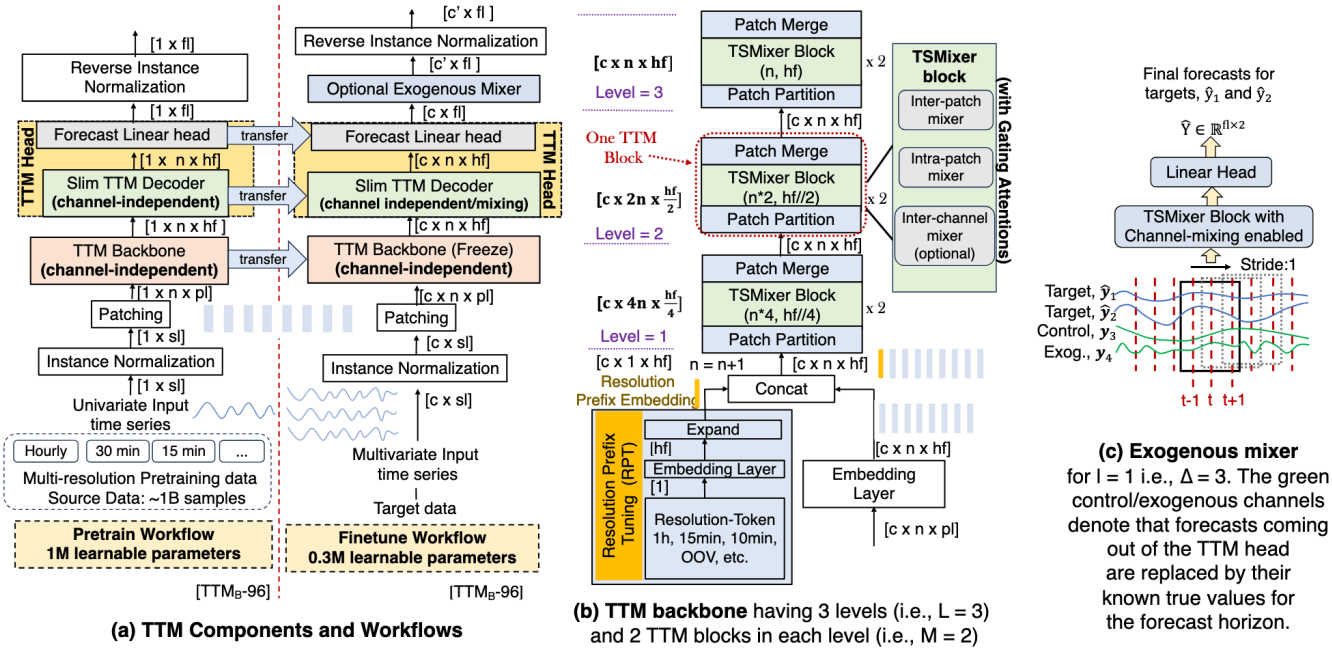

Figure 15.5: Architecture of Tiny Time Mixers.

Tiny Time Mixers (TTM) is a lightweight model (starting at 1M parameters) based on the TSMixer architecture, and developed by IBM Research. The objective was to build a small foundation model that could run on CPU-only machines and reduce computational requirements. It is also a deterministic model and it supports exogenous features.

For more information:

- read the original paper
- see the official repository

Moirai

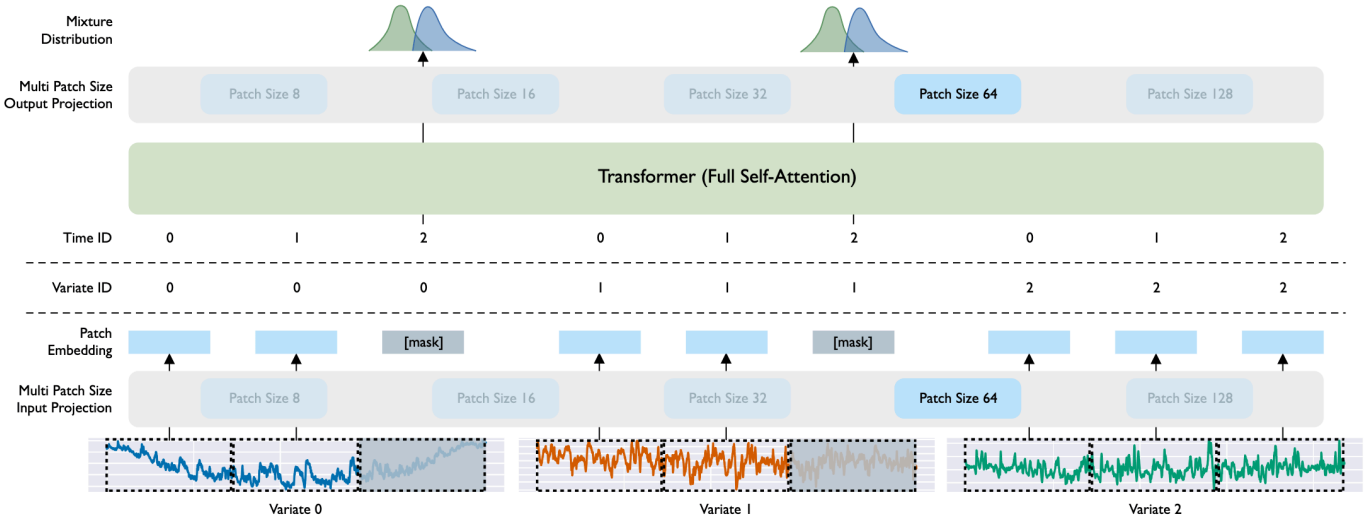

Figure 15.6: Architecture of Moirai.

Moirai is an encoder-only transformer model developed by Salesforce. It is a probabilistic model that also relies on patching the input series. It supports both historical and future exogenous variables. A distinct feature of Moirai is that it uses a mixture distribution, using four different distributions, to output more flexible prediction intervals. Its release also comes with LOTSA, an open archive of time series data with 27B data points.

For more information:

- read the original paper
- see the official repository

Chronos

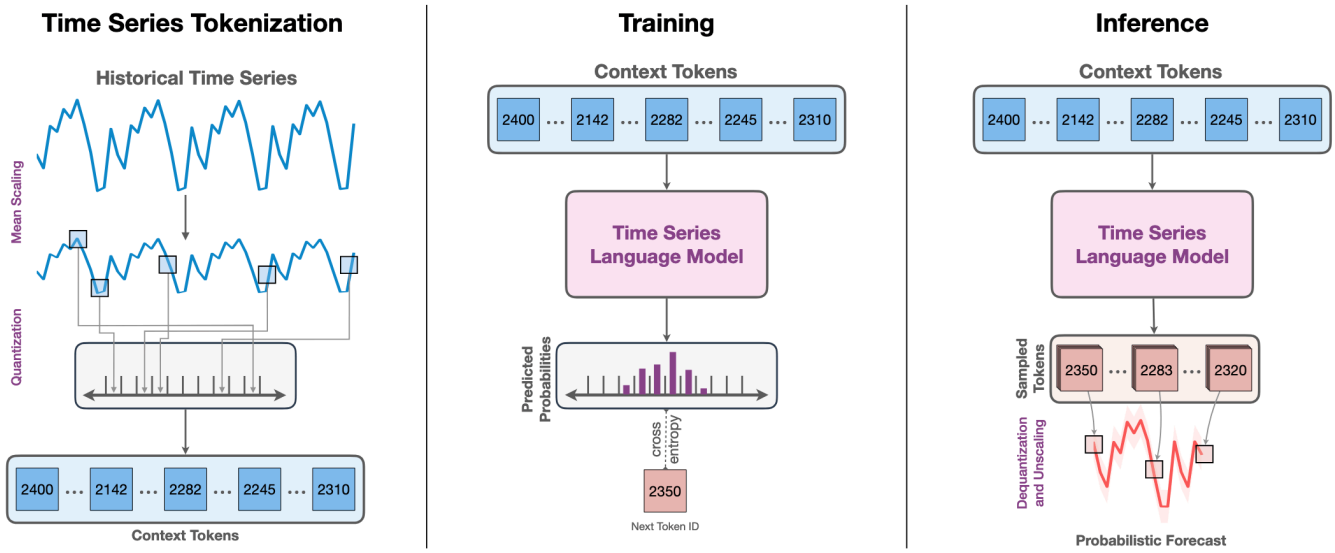

Figure 15.7: Architecture of Chronos.

Chronos is technically a framework to repurpose a large language model (LLM) for time series forecasting developed by Amazon. The researchers have pretrained models based on the T5 family that can be used for zero-shot forecasting. The framework involves tokenizing time series data points, by scaling the data and performing quantization to create a fixed "vocabulary" of time, such that it can be used directly with LLMs. Since LLMs are by default probabilistic models, this results in a probabilistic forecasting model as well.

For more information:

- read the original paper
- see the official repository

15.4 Example: electricity price forecasting

In this example, we predict day-ahead electricity prices with foundation models. We use TimeGPT, which does not require local model weights, and Moirai, one of the few foundation models that supports exogenous features.

The dataset contains data from five electricity markets, each with unique price dynamics, including varying frequencies and occurrences of negative prices, zeros, and price spikes. Since electricity prices depend on exogenous factors, each dataset includes two additional time series: day-ahead forecasts of two significant exogenous factors specific to each market.

This dataset includes hourly prices (`y`), day-ahead forecasts of load (`Exogenous1`), and wind generation (`Exogenous2`). It also includes one-hot encoding (Section 7.4) to indicate the day of the week. For example: Monday (`day_0` = 1), Tuesday (`day_1` = 1), and so on.

For simplicity, we focus on the Nord Pool electricity market (NP), which represents the Nordic countries' exchange.

```
df = pd.read_csv("../data/electricity_short.csv", parse_dates=["ds"])
df
```

	unique_id	ds	y	Exogenous1	Exogenous2	day_0	day_1	day_2	day_3	day_4	day_5	day_6
0	NP	2018-10-15	2.2	34078	1791	1	0	0	0	0	0	0
1	NP	2018-10-15	4.0	33469	1489	1	0	0	0	0	0	0
2	NP	2018-10-15	4.9	33313	1233	1	0	0	0	0	0	0
3	NP	2018-10-15	10.5	33535	1035	1	0	0	0	0	0	0
	...	...	...	...	...	...	...	...	...	...	...	...
1676	NP	2018-12-23	55.6	57002	525	0	0	0	0	0	0	1
1677	NP	2018-12-23	54.0	55990	478	0	0	0	0	0	0	1
1678	NP	2018-12-23	53.9	54336	410	0	0	0	0	0	0	1
1679	NP	2018-12-23	52.3	51679	371	0	0	0	0	0	0	1

TimeGPT is offered by Nixtla as a service and can be used through an API. Nixtla offers free access for academics, students, and researchers. To create an account and generate the API key, visit their website. For specific details on the API, visit the documentation.

```
nixtla_client = NixtlaClient(api_key="YOUR_API_KEY")
preds_df = nixtla_client.forecast(df=df, h=24, level=[80, 90])
preds_df.head()
```

	unique_id	ds	TimeGPT	TimeGPT-hi-80	TimeGPT-hi-90	TimeGPT-lo-80	TimeGPT-lo-90
0	NP	2018-12-24 00:00:00	51.0	52.3	52.5	49.8	49.5
1	NP	2018-12-24 01:00:00	50.1	51.4	51.6	48.9	48.6
2	NP	2018-12-24 02:00:00	49.7	51.2	51.4	48.2	48.0
3	NP	2018-12-24 03:00:00	49.1	50.4	50.5	47.8	47.7
4	NP	2018-12-24 04:00:00	49.6	50.7	50.8	48.6	48.5

```
plot_series(df, preds_df, level=[80],
    max_insample_length=100, xlabel="Hour", ylabel="Price",
    title="Electricity price of the Nord Pool electricity market",
    palette="black_and_blue", rm_legend=False)
```

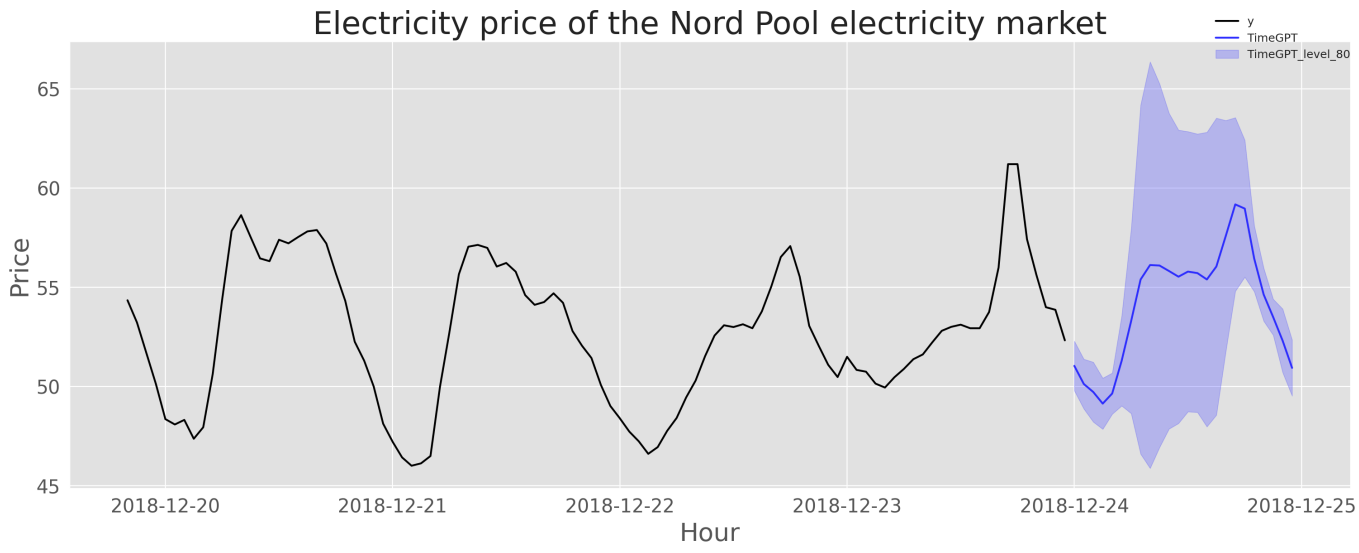

Figure 15.8: Zero-shot forecasting with TimeGPT.

Next, we use cross-validation to evaluate the accuracy of the predictions over the last three windows.

```
cv_preds_df = nixtla_client.cross_validation(df=df, h=24, n_windows=3)
cv_preds_df.head()
```

	unique_id	ds	cutoff	y	TimeGPT
0	NP	2018-12-21 00:00:00	2018-12-20 23:00:00	47.2	47.6
1	NP	2018-12-21 01:00:00	2018-12-20 23:00:00	46.4	47.2
2	NP	2018-12-21 02:00:00	2018-12-20 23:00:00	46.0	46.5

	unique_id	ds	cutoff	y	TimeGPT
3	NP	2018-12-21 03:00:00	2018-12-20 23:00:00	46.1	47.2
4	NP	2018-12-21 04:00:00	2018-12-20 23:00:00	46.5	48.0

```
def transform_evaluation(evaluation):
    return (
        evaluation
        .assign(metric=lambda x: x["metric"].str.upper())
        .drop(columns=["unique_id", "cutoff"])
        .melt(id_vars='metric', var_name='Method')
        .groupby(["metric", "Method"], as_index=False)
        .agg({"value": "mean"})
        .pivot(index="Method", columns="metric", values="value")
        .reset_index().rename_axis("", axis="columns")
    )

evaluation = transform_evaluation(
    evaluate(cv_preds_df, models=["TimeGPT"], metrics=[mae])
)
evaluation
```

	Method	MAE
0	TimeGPT	1.902

Without fine-tuning, TimeGPT achieves an average MAE of 1.902.

Fine-tuning

One important characteristic of foundation models is their ability to be fine-tuned to specific datasets. This can be done with TimeGPT using the `finetune_steps` argument. We can also specify the `finetune_loss` argument to optimize a specific performance metric. In the example, we optimize for the mean absolute error (MAE).

```
cv_finetune_preds_df = nixtla_client.cross_validation(
    df=df, h=24, n_windows=3,
    finetune_steps=15, finetune_loss="mae")
evaluation = transform_evaluation(
    evaluate(cv_finetune_preds_df, models=["TimeGPT"], metrics=[mae])
    .rename(columns={"TimeGPT": "TimeGPT-finetuned"})
)
evaluation
```

	Method	MAE
0	TimeGPT-finetuned	1.895

With finetuning, the MAE improved to 1.895. Finetuning is usually a good approach to improve the performance of a foundation model on a particular dataset.

Incorporating exogenous variables

Until now, the model has used the historical values of the exogenous variables. However, it is beneficial for the model to provide the values of exogenous features over the forecast horizon (see Section 7.4 on predictors for regression models).

```
future_ex_vars_df = pd.read_csv(
  "../data/electricity_future_vars.csv", parse_dates=["ds"]
)
future_ex_vars_df
```

	unique_id	ds	Exogenous1	Exogenous2	day_0	day_1	day_2	day_3	day_4	day_5	day_6
0	NP	2018-12-24	49119	461	1	0	0	0	0	0	0
1	NP	2018-12-24	48115	484	1	0	0	0	0	0	0
2	NP	2018-12-24	47727	497	1	0	0	0	0	0	0
3	NP	2018-12-24	47673	509	1	0	0	0	0	0	0
	...	...	...	...	...	...	...	...	...	...	...
20	NP	2018-12-24	51787	2785	1	0	0	0	0	0	0
21	NP	2018-12-24	51488	2919	1	0	0	0	0	0	0
22	NP	2018-12-24	50928	3119	1	0	0	0	0	0	0
23	NP	2018-12-24	49889	3306	1	0	0	0	0	0	0

```
exog_preds_df = nixtla_client.forecast(
    df=df, X_df=future_ex_vars_df, h=24, level=[80])
columns = exog_preds_df.columns.str.replace("TimeGPT", "TimeGPT-exog")
exog_preds_df = exog_preds_df.set_axis(columns, axis="columns")
exog_preds_df.head()
```

	unique_id	ds	TimeGPT-exog	TimeGPT-exog-hi-80	TimeGPT-exog-lo-80
0	NP	2018-12-24 00:00:00	51.0	53.9	48.0
1	NP	2018-12-24 01:00:00	51.1	54.5	47.6
2	NP	2018-12-24 02:00:00	50.1	53.1	47.1
3	NP	2018-12-24 03:00:00	49.4	51.7	47.1
4	NP	2018-12-24 04:00:00	49.3	50.7	47.8

```
fcst_df = preds_df.merge(exog_preds_df)
plot_series(df, fcst_df,
    max_insample_length=100, xlabel="Hour", ylabel="Price",
    title="Electricity price of the Nord Pool electricity market",
    palette="black_and_2color", rm_legend=False)
```

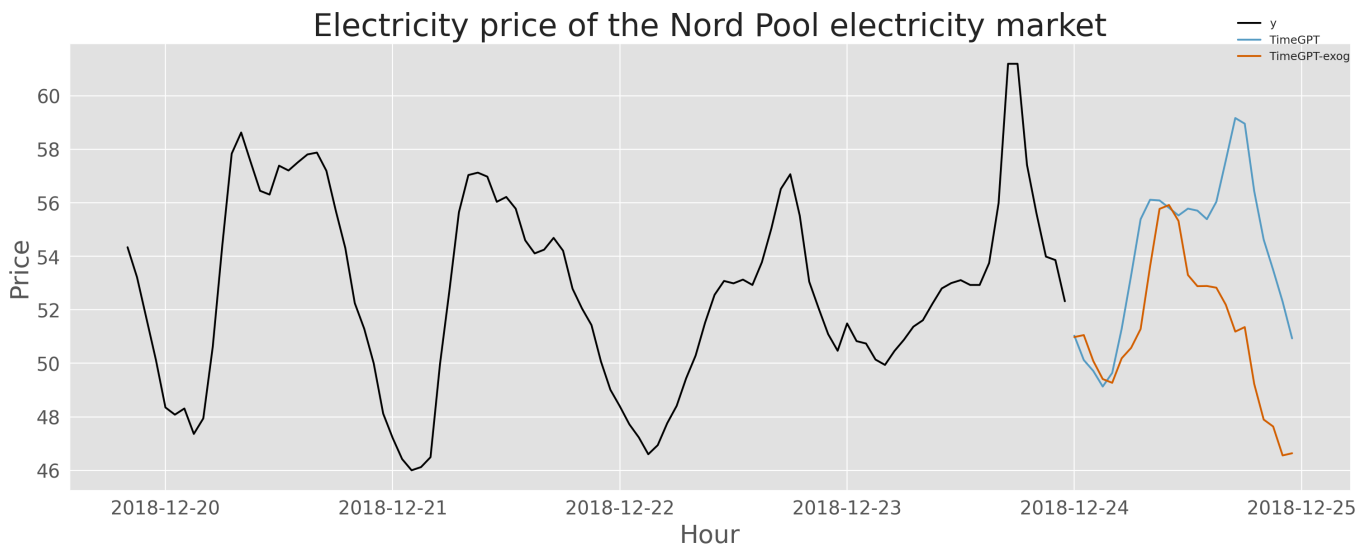

Figure 15.9: Forecasting with exogenous features with TimeGPT.

In the figure above, we can see that by using exogenous features, the model can model variations in the data that were not previously captured.

As mentioned before, Moirai is also a foundation model that supports the use of exogenous variables. First, we start by converting the dataset to a GluonTS dataset.

```
full_df = pd.concat([df, future_ex_vars_df], axis=0)
full_df = full_df.set_index("ds")
ds = PandasDataset.from_long_dataframe(
    full_df,
    target="y",
    item_id="unique_id",
    feat_dynamic_real=full_df.columns.drop(["unique_id", "y"]).tolist(),
)
```

Para manter a consistência, limitamos o horizonte de previsão a 24 etapas temporais no futuro.

```
train, test_template = split(ds, offset=-24)
test_data = test_template.generate_instances(
    prediction_length=24, windows=1, distance=24,
)
```

Em seguida, inicializamos o modelo Moirai. Podemos escolher entre `small`, `base` e `large`. Aqui, vamos usar o `large` modelo.

```
model = MoiraiForecast(
    module=MoiraiModule.from_pretrained(f"Salesforce/moirai-1.0-R-large"),
    prediction_length=24,
    context_length=240,
    patch_size="auto",
    num_samples=50,
    target_dim=1,
    feat_dynamic_real_dim=ds.num_feat_dynamic_real,
    past_feat_dynamic_real_dim=ds.num_past_feat_dynamic_real,
)
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher
rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please
set a HF_TOKEN to enable higher rate limits and faster downloads.
```

Uma vez que o modelo esteja inicializado, podemos usá-lo para previsão zero-shot com as características exógenas disponíveis.

```
predictor = model.create_predictor(batch_size=32)
forecasts = list(predictor.predict(test_data.input))
```

```
fc = forecasts[0]
moirai_preds_df = (
    pd.DataFrame(np.quantile(fc.samples, [0.5, 0.1, 0.9], axis=0).T)
    .set_axis(["Moirai", "Moirai-lo-80", "Moirai-hi-80"], axis="columns")
    .assign(ds=fc.index.to_timestamp())
)
```

Como o modelo Moirai é probabilístico, ele retorna uma distribuição de valores futuros. Portanto, vamos considerar a mediana como a previsão pontual.

```
fcst_df = preds_df.merge(exog_preds_df).merge(moirai_preds_df)
plot_series(df, fcst_df, max_insample_length=100,
    xlabel="Hour", ylabel="Price",
```

```
    title="Electricity price of the Nord Pool electricity market",
    palette="black_and_3color", rm_legend=False)
```

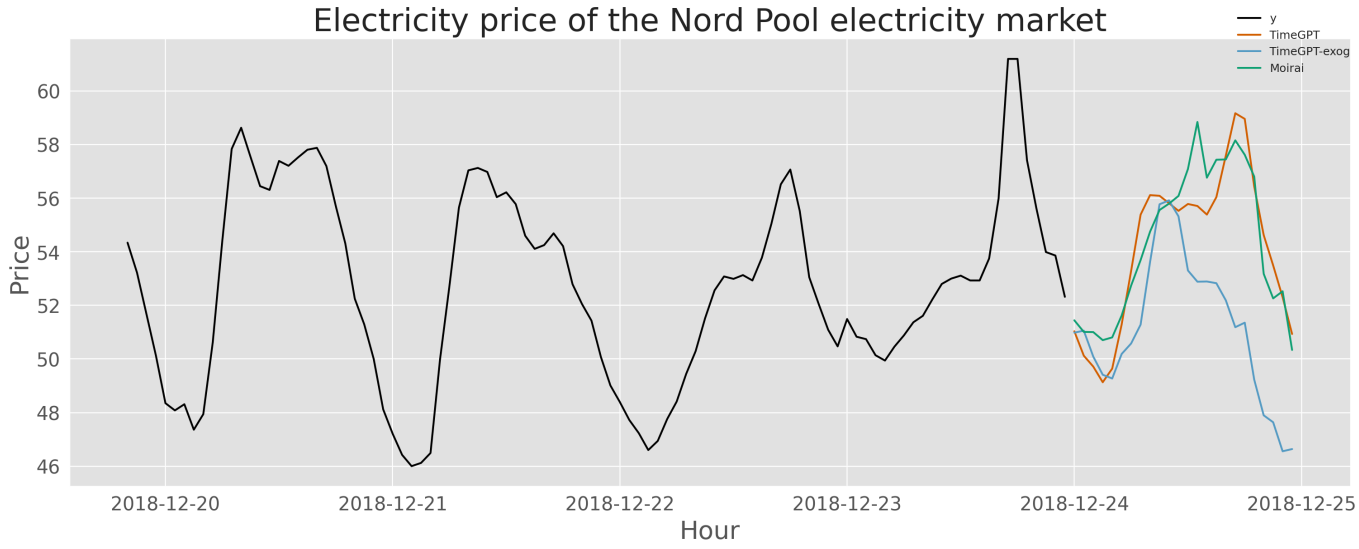

Figura 15.10: Previsão com características exógenas usando TimeGPT e Moirai.

15.5 Leitura complementar

- Um estudo comparativo de Nixtla sobre os modelos fundamentais discutidos neste capítulo está disponível aqui .
- Uma revisão do estado atual e das perspectivas futuras dos LLMs e dos Modelos de Fundação na previsão pode ser encontrada em Bergmeir (2024) .
- Outro benchmark recente da Salesforce é Aksu et al. (2024) .
- Uma pesquisa abrangente sobre modelos de fundação é Liang et al. (2024) .

Se você quiser saber mais sobre um benchmark dos modelos fundamentais discutidos neste capítulo, consulte este experimento aqui .

15.6 Módulos e classes utilizados

NixtlaClient

- `NixtlaClient` Classe - Cliente principal para acessar a API TimeGPT

NeuralPrevisão

- `NeuralForecast` classe - Estrutura para modelos de previsão neural
- `NHITS` Modelo - Interpolação hierárquica neural para modelo de séries temporais

Previsão de utilidades

- `plot_series()` Utilitário - Para criar visualizações de séries temporais
- `evaluate()` Utilidade - Para avaliação de modelos

Publicado pela OTexts usando Quarto . © 2026